Sprint Retrospective Document -- IBMS (Integrated Business Management Suite)

Field	Detail
Project Name	Integrated Business Management Suite (IBMS) v2.0
Team	Shivansh Srivastava (Product Developer), Prahallad Padhan (Product Owner), Ranveer Rai Khare (Scrum Master)
Sprint	Sprint 1
Sprint Duration	2 weeks
Date	April 12, 2026
Document Type	Sprint Retrospective

---

1. Sprint Goal

> *Build a production-grade, AI-first enterprise management platform with real-time dashboards, AI-powered analytics, enterprise security, and multi-environment deployment support.*

Sprint Goal Status: [x] **Achieved**

---

2. Sprint Deliverables Summary

Deliverable	Planned	Delivered	Status
FastAPI backend with 70+ API endpoints	Yes	Yes	[x] Done
JWT authentication + TOTP 2FA	Yes	Yes	[x] Done
RBAC with 5-tier role hierarchy	Yes	Yes	[x] Done
Real-time KPI dashboard (WebSocket)	Yes	Yes	[x] Done
AI Copilot with NL queries	Yes	Yes	[x] Done

Deliverable	Planned	Delivered	Status
Risk scoring & fraud detection engines	Yes	Yes	[x] Done
Compliance engine	Yes	Yes	[x] Done
Budget optimizer	Yes	Yes	[x] Done
Dynamic pricing service	Yes	Yes	[x] Done
Lead scoring service	Yes	Yes	[x] Done
Digital twin simulation	Yes	Yes	[x] Done
Event-driven pub/sub system	Yes	Yes	[x] Done
Background job scheduler (6 jobs)	Yes	Yes	[x] Done
Frontend SPA with Chart.js	Yes	Yes	[x] Done
Docker + Docker Compose setup	Yes	Yes	[x] Done
Kubernetes deployment manifests	Yes	Yes	[x] Done
AWS Terraform infrastructure	Yes	Yes	[x] Done
Unit test suite (pytest)	Yes	Yes	[x] Done
CI/CD pipeline (GitHub Actions)	Yes	Yes	[x] Done

3. What Went Well [x]

#	Item	Impact
1	FastAPI choice for backend	Async support, auto-generated OpenAPI docs, and Pydantic validation accelerated development significantly.
2	Redis with in-memory fallback	The graceful degradation pattern ensured the app runs without Redis, simplifying local development and testing.
3	Monolithic architecture for v1	Keeping all modules in a single process reduced operational complexity and made debugging straightforward.
4	Comprehensive security from Day 1	Building JWT + 2FA + RBAC + CSRF + rate limiting + device binding from the start avoided retrofitting security later.
5	WebSocket-based real-time updates	Live KPI push every 15 seconds delivered an engaging dashboard experience without polling overhead.
6	Mock-based test strategy	Mocking Frappe ORM allowed unit tests to run independently without a database, enabling fast CI execution.

#	Item	Impact
7	Multi-tier deployment readiness	Docker Compose, K8s, and Terraform configs were built alongside the application, not as afterthoughts.
8	Vanilla JS frontend with no build step	Eliminated build toolchain complexity; changes to the frontend were immediately visible on refresh.

4. What Could Be Improved ?

#	Item	Impact	Action Plan
1	Low automated test coverage	Only 5 unit tests across 2 test files; many modules untested	Add tests for all services, security middleware, and API endpoints in Sprint 2
2	No E2E/integration test suite	Frontend flows (login, dashboard, charts) are untested	Introduce Playwright or Cypress E2E test framework
3	No database integration tests	All DB interactions mocked; schema issues may slip through	Add Frappe test bench integration or SQLite-based test DB
4	Missing .env template	New developers must guess required environment variables	Create `.env.example` with documented defaults
5	Some endpoints lack authentication	Business endpoints like `/api/risk/score` and `/api/fraud/detect` don't require auth	Review and add Bearer auth to sensitive endpoints
6	Frontend code in single JS file	`app.js` is large and handles auth, API, dashboard, all pages	Split into modular ES6 imports or a lightweight framework
7	No API versioning	All routes under `/api/` without version prefix	Introduce `/api/v1/` prefix for forward compatibility
8	No load testing	No performance baselines established	Add Locust or k6 load testing scripts

5. What to Start Doing ?

#	Item	Rationale
1	API endpoint authentication audit	Secure all business-critical endpoints with Bearer JWT
2	Automated E2E testing pipeline	Catch frontend regressions before they reach production

#	Item	Rationale
3	Database migration scripts	Versioned schema management for production deployments
4	API documentation with examples	Auto-generate OpenAPI spec with request/response examples
5	Dependency vulnerability scanning	Add `pip-audit` or Snyk to CI/CD pipeline
6	Performance monitoring baselines	Establish p50/p95/p99 latency targets for critical endpoints
7	Feature flags system	Enable gradual rollout of new AI/ML features

6. What to Stop Doing ?

#	Item	Rationale
1	Hardcoded secrets in source code	`JWT_SECRET` has a default in `server.py`; must always come from environment
2	Skipping tests for "simple" modules	Budget optimizer and compliance engine have no tests despite being critical
3	Manual integration testing	API endpoint verification was done via curl; should be automated

7. Sprint Metrics

Metric	Value
Planned Features	19
Delivered Features	19
Completion Rate	100%
API Endpoints Delivered	70+
Services Implemented	9
Background Jobs Created	6
Automated Unit Tests	5
Manual Test Cases	9

Metric	Value
Security Layers Implemented	7 (JWT, 2FA, RBAC, CSRF, rate limit, device binding, audit)
Deployment Configs	3 (Docker, K8s, AWS Terraform)
Critical Bugs Found	0
Tech Debt Items Identified	8

8. Risks & Mitigations

Risk	Likelihood	Impact	Mitigation
Security vulnerability in unauthenticated endpoints	Medium	High	Auth audit + endpoint protection in Sprint 2
Test coverage too low for refactoring confidence	High	Medium	Target 60%+ unit test coverage in Sprint 2
Frontend scalability with single JS file	Medium	Medium	Modularize or migrate to component framework
Redis dependency for production workloads	Low	Medium	In-memory fallback already implemented
Terraform state management at scale	Low	High	Introduce remote state (S3 + DynamoDB locking)

9. Action Items for Sprint 2

#	Action Item	Owner	Priority	Target
1	Add unit tests for all 9 services	Shivansh Srivastava	High	Sprint 2
2	Add Bearer auth to unprotected business APIs	Shivansh Srivastava	High	Sprint 2
3	Create `.env.example` with all config variables	Shivansh Srivastava	Medium	Sprint 2
4	Set up Playwright E2E testing framework	Shivansh Srivastava	Medium	Sprint 2
5	Introduce `/api/v1/` API versioning	Shivansh Srivastava	Medium	Sprint 2
6	Add `pip-audit` to CI pipeline	Shivansh Srivastava	Medium	Sprint 2
7	Modularize `app.js` into ES6 modules	Shivansh Srivastava	Low	Sprint 3

#	Action Item	Owner	Priority	Target
8	Add Locust load testing scripts	Shivansh Srivastava	Low	Sprint 3

10. Team Feedback

> *Overall Sprint Sentiment: Positive -- the team successfully delivered a feature-complete enterprise platform in a single sprint. The architecture is solid and extensible. The primary concern going into Sprint 2 is increasing test coverage and hardening the security posture of public-facing endpoints.*

Team Members

Name	Role
Shivansh Srivastava	Product Developer
Prahallad Padhan	Product Owner
Ranveer Rai Khare	Scrum Master

Document Version 1.0 -- Sprint 1 Review